2023 IEEE 12th International Conference on Intelligent Data Acquisition and Advanced Computing Systems: Technology and Applications (IDAACS) | 979-8-3503-5805-6/23/$31.00 ©2023 IEEE | DOI: 10.1109/IDAACS58523.2023.10348763

Ant Colony Optimization for Resource Allocation in Cloud Computing Environments

Taras Kniazhyk, Oleksandr Muliarevych
Lviv Polytechnic National University, 12, Bandera Str, Lviv, 79013, Ukraine,
taras.kniazhyk.mkiks.2022@lpnu.ua, oleksandr.v.muliarevych@lpnu.ua

***Abstract*—This article delves into cloud computing technologies, addressing the challenges and concerns that arise in this domain, specifically focusing on resource allocation. We investigate and compare three optimization algorithms—Particle Swarm Optimization (PSO), Ant Colony Optimization (ACO), and Genetic Algorithm (GA)—in the context of their applications, objectives, and mechanisms of action. Additionally, the article highlights the significance of each algorithm in optimizing cloud resources and discusses their advantages and drawbacks.**

For cloud computing with higher complexity, it becomes increasingly important to develop efficient strategies for managing and allocating resources, which in turn helps organizations control costs and improve performance. Prepared comparative analysis aims to provide a comprehensive understanding of the three algorithms, equipping decision-makers with the knowledge to select the most suitable optimization method for their specific cloud computing needs and challenges.

***Keywords*—ACO, PSA, GA, resource allocation, cloud computing.**

I. Introduction and State of the Art

Cloud computing has emerged as a critical technology, offering scalable and cost-effective solutions for businesses. However, managing resources and controlling costs remain significant challenges. Existing optimization algorithms such as Particle Swarm Optimization (PSO), Ant Colony Optimization (ACO), and Genetic Algorithms (GA) address resource allocation issues. **The aim of this research** is to demonstrate how nature-inspired optimization algorithms could be adjusted to apply for resource allocation problem solving within cloud computing environments and investigate their efficiency.

Understanding and managing cloud expenses can be a daunting task for organizations, especially as costs may unexpectedly increase. While cloud service providers offer scalability benefits, without the proper tools and services, expenses can quickly get out of hand. With 73% of cloud decision-makers facing similar issues, cloud optimization becomes essential [1].

This comprehensive guide elaborates on cloud optimization, its importance, examples, and tools for optimizing cloud expenditures. Readers will gain a clear understanding of the advantages of cloud optimization and how to begin implementing cost-saving measures. Cloud optimization involves selecting, provisioning, and right-sizing resources spent on specific cloud features to eliminate waste and enhance performance.

In a DevOps context, cloud optimization aims to efficiently allocate resources among various use cases. The definition of cloud optimization may vary among companies depending on their applications. An effective cloud optimization strategy can identify areas for improvement and maximize the benefits of cloud migration and IT spending. For those already deployed in the cloud, the term "cloud management" is often encountered [2].

Objectives and Methodology. Our main objective is to develop an effective resource allocation strategy using the Ant Colony Optimization algorithm. We compared ACO with PSO and GA, focusing on their operating principles, typical tasks, and efficiency in cloud computing applications.

II. Resource Allocation in Cloud Computing

Cloud computing refers to the delivery of hosted services over the internet and is categorized into three types: Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS) [3]. Clouds can be public or private, with the goal of providing easy, scalable access to computing resources and IT services.

Cloud infrastructure consists of hardware and software components needed for implementing a cloud computing model. Three general service delivery categories include: IaaS, PaaS and SaaS.

IaaS: Providers like Amazon Web Services offer virtual server instances, storage, and APIs for workload migration to a virtual machine. IaaS is similar to a remote data center for business users.

PaaS: Cloud providers host development tools on their infrastructure, accessible through APIs, web portals, or gateway software. Examples include Salesforce's Lightning Platform and Google App Engine.

SaaS: A distribution model that delivers software applications over the internet, accessible from any location using a computer or mobile device. Examples include Microsoft 365 for productivity and email services.

Cloud computing offers several benefits such as cost reduction, pay-per-use, broad network access, multi-

Authorized licensed use limited to: BMS College of Engineering. Downloaded on November 05,2026 at 10:55:08 UTC from IEEE Xplore. Restrictions apply.

tenancy, resource pooling, and reliability. Cloud providers continually expand their services, resulting in greater usability, reliability, and scalability compared to traditional compute and storage instances. Serverless computing, for example, allows developers to create code executed by the cloud provider in response to events, eliminating the need to manage servers or instances.

Cloud computing offers businesses and clients an on-demand computing environment for leasing resources. The primary goal for both cloud consumers and suppliers is to allocate cloud resources effectively and profitably [2]. These resources are often limited, and providers must distribute them within the cloud environment's constraints while meeting the needs of cloud applications.

Resource allocation is a critical component of cloud computing, as its efficiency directly impacts the performance of the entire cloud environment [4]. Challenges in resource allocation include cost efficiency, response time, reallocation, computational performance, and task scheduling. Cloud computing consumers aim to complete tasks at the lowest possible cost. Resource allocation entails providing services and storage space for specific tasks requested by users, utilizing various allocation strategies. These strategies involve coordinating cloud provider activities when allocating scarce cloud resources and addressing cloud applications' requirements.

Cloud users and providers, the two parties in a cloud computing environment, have distinct objectives. Providers seek to maximize resource utilization and profits, while users aim to reduce expenses without compromising performance. Various strategies help maintain a balance between resource allocation and cost, preventing over-provisioning, under-provisioning, resource fragmentation, and resource contention.

Resource allocation in the cloud has some limitations:

- Cloud users rent resources from remote servers but lack control over them, making migration and searching for better data storage challenging.
- Data transfer between providers can pose risks, with data stored in the public cloud being vulnerable to phishing and hacking attacks. Malware can spread rapidly due to interconnected servers.
- In-depth knowledge is necessary for mastering resource allocation and management since cloud service providers hold extensive information about cloud operations.
- For using peripheral devices like scanners and printers with the cloud, local software installation is needed, though networked peripherals work well with the cloud.

Enterprises continue to rely on cloud computing for various use cases, including increasing efficiency, optimizing costs, ensuring data security, and storing unlimited data. Knowledge of resource allocation is increasingly essential for cloud users seeking to optimize their cloud usage costs. While many strategies exist for allocating resources in the cloud, the most effective method is the one that satisfies cloud users and maximizes income for cloud service providers.

III. Adjustment of Optimization Methods to solve Resource Allocation Problem

Before Most local optimization algorithms are gradient based. As the name suggests, these techniques utilize gradient information to find the optimal solution. Gradient-based algorithms are widely employed for solving various optimization problems in engineering due to their efficiency (regarding the number of function evaluations needed to find the optimum), capability to solve problems with numerous design variables, and minimal requirement for problem-specific parameter tuning. However, these algorithms also have several drawbacks, such as locating only local optima, difficulty solving discrete optimization problems, complexity in implementation, and susceptibility to numerical noise. In general, these algorithms are well-established and extensively discussed [5].

The discussion of local search techniques would be incomplete without mentioning non-gradient-based local search algorithms, such as Powell's method and the Nelder-Mead simplex algorithm. Both algorithms can solve non-linear, unconstrained optimization problems. Powell's method is founded on the concept of conjugate directions, while the Nelder-Mead algorithm employs a simplex and a set of straightforward rules that reflect the worst vertex through the simplex's centroid.

Evolutionary optimization algorithms [6] have gained popularity over the past couple of decades. Unlike local techniques that update a single design point (typically using gradient information) from one iteration to the next, these algorithms do not require gradient information and generally use a set of design points (commonly referred to as a population) to identify the optimal design. Inspired by natural phenomena, these methods offer advantages such as robustness, an increased likelihood of finding a global or near-global optimum, ease of implementation, and suitability for discrete optimization problems. However, they also have significant drawbacks, including high computational costs, poor constraint-handling abilities, problem-specific parameter tuning, and limited problem size. Now let’s describe multiple nature-inspired alogirhtms that were tested in scope of developed computer system: PSO, ACO and GA.

A. Particle Swarm Optimization Algorithm

Particle Swarm Optimization (PSO) is a bio-inspired algorithm that offers a simple approach to searching for an optimal solution within the solution space. Distinct from other optimization algorithms, PSO only requires the objective function and is not dependent on the gradient or any differential form of the objective. Additionally, it has very few hyperparameters [7].

Authorized licensed use limited to: BMS College of Engineering. Downloaded on November 05,2026 at 10:55:08 UTC from IEEE Xplore. Restrictions apply.

To mathematically model the PSO principles and enable the swarm to find the global minimum of a fitness function we need to consider the following:

- Each particle in Particle Swarm Optimization has an associated position, velocity, and fitness value.
- Each particle keeps track of its "particle bestFitness value" and "particle bestFitness position".
- A record of "global bestFitness position" and "global bestFitness value" is maintained.

The Fig. 1 is represented the data structure used to store the swarm population. This data structure holds information about each particle's position, velocity, and fitness value, as well as their best individual positions and the global best position.

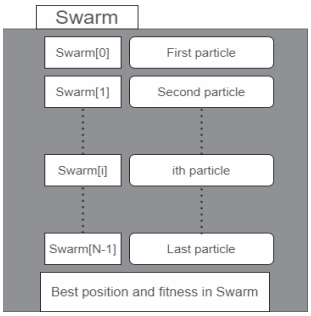


Figure 1. Data structure to store Swarm population.

The Fig. 2 illustrates the data structure used to store the i-th particle of the swarm.

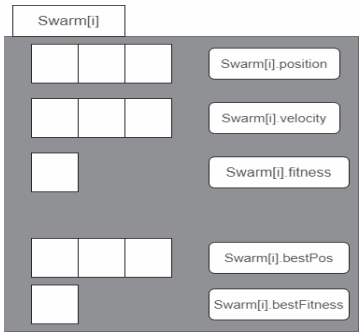


Figure 2. Data structure to store i-th particle of Swarm.

This data structure captures the details specific to the individual particle, such as its position, velocity, and fitness value.

Here is the proposed approach used to adjust PSO for the resource allocation problem solving:

Step 1. Initialization:

- Define the number of particles in the swarm N.
- For each particle i, randomly initialize the position representing a potential solution to resource allocation.
- The position vector might represent how different resources (CPU, memory) are allocated to tasks. Initialize the velocity for each particle.
- Evaluate the fitness of each particle using a predefined objective function – the cost of resources.

Step 2. Iterative Optimization running in two-level cycle with max iteration criteria and per each particle:

- Update the personal best position (pBest) if the current position has a better fitness and udate the global best position (gBest) among all particles.
- Update the velocity of the particle.
- Update the position of the particle.
- If the new position is better than the pBest, update the pBest.
- Update the global best position (gBest) among all particles.

Step 3. Post Optimization: The gBest position represents the optimized resource allocation solution, using it we apply result allocation in the cloud environment. Try to evaluate current resources and consider dynamic changes.

A remarkable aspect of PSO is the presence of a stable topology, where particles can communicate with one another and enhance the learning rate to achieve the global optimum. The metaheuristic nature of this optimization algorithm offers numerous opportunities, as it iteratively seeks to improve a candidate solution by optimizing the problem at hand.

B. Ant Colony Optimization

Ant Colony Optimization (ACO) [8] is a probabilistic technique for finding optimal paths based on the foraging behavior of ants seeking a path between their colony and food sources. Initially applied to the traveling salesman problem, ACO has since been utilized for various complex optimization problems [9]. Ants are social insects that communicate via pheromone trails, with higher pheromone levels increasing the probability of a path being chosen by other ants [10].

To understand ACO's application to the traveling salesman problem (TSP) [8], consider the fully connected construction graph, with vertices representing cities and edge lengths proportional to the distances between cities. Pheromone and heuristic values are associated with graph edges, with ants constructing solutions by probabilistically choosing edges based on these values. Pheromone values are updated after all ants have completed their tour, with the procedure being repeated until a termination criterion is met. Ant Colony Optimization is a population-based metaheuristic that can be used to find approximate solutions to difficult optimization problems. In resource allocation, ACO can be used to find a path through graphs that represent different resource allocation options. Here is the proposed approach used to adjust ACO for the resource allocation problem solving:

Step 1. Initialization: Define the number of ants in the colony N. Place the ants on random positions. Initialize the pheromone levels on the paths.

Step 2. Construct Ant Solutions running in cycle two procedures:

Authorized licensed use limited to: BMS College of Engineering. Downloaded on November 05,2026 at 10:55:08 UTC from IEEE Xplore. Restrictions apply.

- Construct a solution by moving the ant from state to state within the feasible solution space.
- At each step, apply a stochastic local search procedure biased by pheromone levels to simulate decision making.

Step 3. Pheromone Update:

- Evaporate some portion of the pheromone on all paths.
- For each ant, deposit pheromone on the paths it took based on the quality of its solution.

Step 4. Daemon Actions - this phase is used to evaluate currently available resources and consider dynamic changes.

Step 5. Convergence Check – check for convergence criteria. If not converged, go to Step 2.

Step 6: Solution Construction – construct the resource allocation solution from the highest pheromone path.

C. *Genetic Algorithm*

Define Genetic algorithm (GA) [11] is optimization method based on natural selection that solve constrained and unconstrained problems. GA modifies a population of individual solutions over successive generations, evolving toward an optimal solution [12].

Three main rules [13] are used in GA to create the next generation: 1) Selection rules. Parents are stochastically selected based on their fitness scores. 2) Crossover rules. Two parents are combined to form children. 3) Mutation rules. Random changes are applied to individual parents to form children.

Here is the proposed approach used to adjust GA for the resource allocation problem solving:

Step 1. Initialization: Generate an initial population of chromosomes, where each chromosome is a feasible resource allocation. Evaluate the fitness of each chromosome in the population.

Step 2. Selection – select chromosomes to form a new population; selection should favor fitter solutions.

Step 3. Crossover: with a crossover probability, crossover the pairs of chromosomes randomly chosen from the population. This is done to reproduce a new population of chromosomes.

Step 4. Mutation: with mutation probability, mutate the new population at each position.

Step 5. Evaluation – evaluate the individual fitness of each chromosome in the current population.

Step 6. Termination Check – if maximum number of generations has been reached or dynamic changes appeared we need to terminate. In case result is not valid, or calculation not finished go back to Step 2.

Step 7. Solution Construction - the best chromosome represents the optimized resource allocation solution, apply this allocation in the cloud environment.

Genetic algorithms have parallel capabilities, making them suitable for optimizing various problems, including discrete functions and multi-objective problems. They do not require derivative information and can generate multiple optimal results. However, classical GA is not efficient for solving simple problems and may face computational challenges due to repetitive fitness value calculations.

IV. Case Study

In this section, we present an empirical evaluation of three described previously optimization algorithms: Ant Colony Optimization (ACO), Particle Swarm Optimization (PSO) and Genetic Algorithm (GA), to address resource allocation problems in cloud computing environments. The performance of these algorithms is compared in a well-controlled experimental setup.

The Fig.3 demonstrates the performance comparison of Ant Colony Optimization (ACO), Particle Swarm Optimization (PSO), and Genetic Algorithm (GA) in solving resource allocation problems in cloud computing environments.

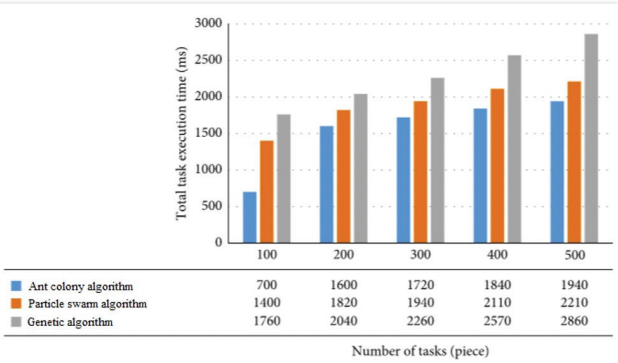


Figure 3. Performance test results of ACO, PSA, GA algorithms.

The experiments were conducted in a virtualized cloud environment hosted on Amazon Web Services (AWS) [14]. The cloud infrastructure consisted of a cluster of EC2 instances. The chosen instance types were a mix of compute-optimized (c5.large) and memory-optimized (r5.large) instances, ensuring heterogeneity typical in real-world cloud environments [15,16]. Each c5.large instance was equipped with 2 vCPUs and 4GB of RAM, whereas each r5.large instance had 2 vCPUs and 16GB of RAM.

The workload comprised various computational tasks typical in cloud computing such as data processing and graphical data storage processing. Each task had diverse resource requirements and QoS constraints [17].

The objective was to minimize the makespan (total completion time) and cost while ensuring a minimum level of performance for each task. Therefore, a multi-objective fitness function was used for evaluation.

Test configurations for agorithms:

1) For the Ant Colony Optimization algorithm, we have used next parameters: number of ants = 20, evaporation rate = 0.1, pheromone influence: 1.0; heuristic influence = 1.0.

2) For the PSO algorithm, we have used next parameters: number of particles = 40; inertia weight = 0.729; cognitive constant = 1.49445; social constant = 1.49445.

Authorized licensed use limited to: BMS College of Engineering. Downloaded on November 05,2026 at 10:55:08 UTC from IEEE Xplore. Restrictions apply.

3) For the GA algorithm, we have used next parameters: population size = 50; crossover probability = 0.8; mutation probability = 0.02.

For each algorithm, 30 independent runs were conducted. In each run, the algorithm attempted to allocate resources to a set of tasks. The tasks and their requirements were generated based on a combination of real traces and synthetic workloads to represent a wide range of scenarios.

The tasks considered in this test scenario could include various computational workloads, such as data processing, machine learning model training, scientific simulations, and large-scale optimization problems. As these tasks often require substantial computational resources and have diverse requirements, effective resource allocation is crucial to ensure efficient execution, user satisfaction, and optimal resource utilization.

CONCLUSIONS

In article was demonstrated how nature-inspired optimization methods could be adjusted to solve resource allocation problems. Our research establishes the effectiveness of the Ant Colony Optimization algorithm for resource allocation in cloud computing environments. Despite its computational expense and parameter tuning requirements, ACO delivers superior performance compared to PSO and GA. Future work includes refining the algorithm to minimize its disadvantages and exploring parallelization techniques to enhance real-time application.

By analyzing the experimental results, it is evident that the ACO algorithm outperforms PSO and GA in the context of the selected tasks, particularly as the number of tasks increases from 50% to 11% comparing to PSO and from 60% to 21% comparing to GA. The proposed ACO-based method for cloud computing resource allocation has the potential to cater to a wide range of tasks while maintaining efficiency, flexibility, and adaptability in dynamic and complex cloud computing environments.

REFERENCES

[1] B. Dewangan, T. Choudhury, T. Toe, B. Singh, N. Nhu, R. Tomar, *Cloud Autonomic Computing in Cloud Resource Management in Industry 4.0*. Switzerland: Springer, 2021, pp. 123–195.

[2] N. Sehgal, P. Bhatt., J. Acken, *Cloud Computing with Security*, Concepts and Practices. Second edition. Switzerland: Springer, 2020, pp. 75–109.

[3] P. Raj, S. Vanga, A. Chaudhary, Cloud-native Computing: How to Design, Develop, and Secure Microservices and Event-Driven Applications. John Wiley & Sons, 2022. – pp. 129–163.

[4] A. Singh, L. Indrusiak, P. Dziurzanski, *Dynamic Resource Allocation in Embedded, High-Performance and Cloud Computing*. e-book, Denmark: River Publishers, 2022. – 178 p.

[5] M. J. Kochenderfer, T. A. Wheeler, *Algorithms for Optimization*. United Kingdom: MIT Press, 2019. – pp. 125-189.

[6] Q. H. Badar, Altaf. *Evolutionary Optimization Algorithms*. United States, CRC Press, 2021. – 273 p.

[7] J. Cai, P. Peng, X. Huang and B. Xu, "A Hybrid Multi-phased Particle Swarm Optimization with Sub Swarms," 2020 International Conference on Artificial Intelligence and Computer Engineering (ICAICE), Beijing, China, 2020, pp. 104-108.

[8] O. Muliarevych, "Solving dynamic assymetrical Travelling Salesman Problem in conditions of partly unknown data", Lviv-Slavsk, Lviv Polytechnik Publ., TCSET'2016 vol.1, 2016. pp. 446–448.

[9] O. Kozlov, "Information technology for designing rule bases of fuzzy systems using ant colony optimization," *International Journal of Computing*, vol. 20, issue 4, pp. 471-486, 2021. https://doi.org/10.47839/ijc.20.4.2434

[10] S. Jun, N. Yatskiv, A. Sachenko and V. Yatskiv, "Improved method of ant colonies to search independent data transmission routes in WSN," 2012 IEEE 1st International Symposium on Wireless Systems (IDAACS-SWS), Offenburg, Germany, 2012, pp. 52-57, doi: 10.1109/IDAACS-SWS.2012.6377632.

[11] E. Arianyan, D. Maleki, A. Yari and I. Arianyan, "Efficient resource allocation in cloud data centers through genetic algorithm," 6th International Symposium on Telecommunications (IST), Tehran, Iran, 2012, pp. 566-570.

[12] S. Liu and Z. Li, "A modified genetic algorithm for community detection in complex networks," 2017 International Conference on Algorithms, Methodology, Models and Applications in Emerging Technologies (ICAMMAET), Chennai, India, 2017, pp. 1-3, doi: 10.1109/ICAMMAET.2017.8186747.

[13] L. Yichen, L. Bo, Z. Chenqian and M. Teng, "Intelligent Frequency Assignment Algorithm Based on Hybrid Genetic Algorithm," 2020 International Conference on Computer Vision, Image and Deep Learning (CVIDL), Chongqing, China, 2020, pp. 461-467, doi: 10.1109/CVIDL51233.2020.00-50.

[14] O. Muliarevych, "Acceptance and shipping warehouse zones calculation using serverless approach," 2022 12th International Conference on Dependable Systems, Services and Technologies (DESSERT), Athens, Greece, 2022, pp. 1-6, (doi: 10.1109/DESSERT58054.2022.10018786).

[15] A. M. Sampaio and J. G. Barbosa, "Enhancing Reliability of Compute Environments on Amazon EC2 Spot Instances," 2019 International Conference on High Performance Computing & Simulation (HPCS), Dublin, Ireland, 2019, pp. 708-715, doi: 10.1109/HPCS48598.2019.9188116.

[16] N. Ekwe-Ekwe and A. Barker, "Location, Location, Location: Exploring Amazon EC2 Spot Instance Pricing Across Geographical Regions," 2018 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID), Washington, DC, USA, 2018, pp. 370-373, doi: 10.1109/CCGRID.2018.00059.

[17] D. Ardagna and B. Pernici, "Global and local QoS constraints guarantee in Web service selection," IEEE International Conference on Web Services (ICWS'05), Orlando, FL, USA, 2005, pp. 806, doi: 10.1109/ICWS.2005.66.

Authorized licensed use limited to: BMS College of Engineering. Downloaded on November 05,2026 at 10:55:08 UTC from IEEE Xplore. Restrictions apply.